

LISTA SIMPLE

```
import java.util.ArrayList;
public class EjemploLista {
    public static void main(String[] args) {
        // Crear una lista de tipo String
        ArrayList<String> listaNombres = new ArrayList<>();
        // Agregar elementos a la lista
        listaNombres.add("Ana");
        listaNombres.add("Luis");
        listaNombres.add("Pedro");
        listaNombres.add("Maria");

        // Imprimir los elementos de la lista
        System.out.println("Lista de nombres:");
        for (String nombre : listaNombres) {
            System.out.println(nombre);
        }

        // Acceder a un elemento por índice
        System.out.println("El primer nombre de la lista es: " + listaNombres.get(0));

        // Modificar un elemento de la lista
        listaNombres.set(1, "Carlos");
        System.out.println("Lista de nombres después de la modificación:");
        for (String nombre : listaNombres) {
            System.out.println(nombre);
        }

        // Eliminar un elemento de la lista
        listaNombres.remove("Pedro");
        System.out.println("Lista de nombres después de la eliminación:");
        for (String nombre : listaNombres) {
            System.out.println(nombre);
        }

        // Obtener el tamaño de la lista
        System.out.println("Tamaño de la lista: " + listaNombres.size());
    }
}
```

LISTA DOBLEMENTE ENLAZADA

```
import java.util.LinkedList;

class Alumno {
    private String nombre;
    private int numeroControl;

    public Alumno(String nombre, int numeroControl) {
        this.nombre = nombre;
        this.numeroControl = numeroControl;
    }

    public String getNombre() {
        return nombre;
    }

    public int getNumeroControl() {
        return numeroControl;
    }

    @Override
    public String toString() {
        return "Nombre: " + nombre + ", Número de Control: " + numeroControl;
    }
}

public class EjemploListaDobleEnlazada {
    public static void main(String[] args) {
        // Crear una lista doblemente enlazada de objetos Alumno
        LinkedList<Alumno> listaAlumnos = new LinkedList<>();

        // Agregar alumnos a la lista
        listaAlumnos.add(new Alumno("Ana", 123456));
        listaAlumnos.add(new Alumno("Luis", 234567));
        listaAlumnos.add(new Alumno("Pedro", 345678));
        listaAlumnos.add(new Alumno("Maria", 456789));

        // Imprimir los alumnos de la lista
        System.out.println("Lista de alumnos:");
        for (Alumno alumno : listaAlumnos) {
            System.out.println(alumno);
        }

        // Acceder al primer y último elemento
        System.out.println("\nPrimer alumno de la lista: " + listaAlumnos.getFirst());
        System.out.println("Último alumno de la lista: " + listaAlumnos.getLast());

        // Modificar un alumno
        listaAlumnos.set(1, new Alumno("Carlos", 234567));
        System.out.println("\nLista de alumnos después de la modificación:");
        for (Alumno alumno : listaAlumnos) {
            System.out.println(alumno);
        }

        // Eliminar un alumno de la lista
        listaAlumnos.remove(2); // Elimina al alumno en el índice 2
        System.out.println("\nLista de alumnos después de la eliminación:");
        for (Alumno alumno : listaAlumnos) {
            System.out.println(alumno);
        }

        // Tamaño de la lista
        System.out.println("\nTamaño de la lista: " + listaAlumnos.size());
    }
}
```

```
}
```

En Java se utiliza para indicar que un método está sobrescribiendo (o anulando) un método de su clase padre (una clase que está heredando o una interfaz que está implementando). Es una forma de asegurar que estamos modificando correctamente el método que queremos en lugar de crear uno nuevo por error.

En Java, las **listas doblemente enlazadas** se pueden implementar mediante una clase personalizada, dado que Java no tiene punteros explícitos como C++ (se usan referencias de objetos). Aquí tienes un ejemplo básico de una lista doblemente enlazada en Java, donde cada nodo apunta a su nodo anterior y su nodo siguiente.

```
class Nodo {
    int dato;
    Nodo siguiente;
    Nodo anterior;

    public Nodo(int dato) {
        this.dato = dato;
        this.siguiente = null;
        this.anterior = null;
    }
}

class ListaDoble {
    private Nodo cabeza;
    private Nodo cola;

    public ListaDoble() {
        this.cabeza = null;
        this.cola = null;
    }

    // Agregar al final de la lista
    public void agregar(int valor) {
        Nodo nuevo = new Nodo(valor);

        if (cabeza == null) {
            cabeza = cola = nuevo;
        } else {
            cola.siguiente = nuevo;
            nuevo.anterior = cola;
            cola = nuevo;
        }
    }

    // Eliminar un nodo específico
    public void eliminar(int valor) {
        Nodo actual = cabeza;

        while (actual != null) {
            if (actual.dato == valor) {
                if (actual.anterior != null) {
                    actual.anterior.siguiente = actual.siguiente;
                } else {
                    cabeza = actual.siguiente;
                }

                if (actual.siguiente != null) {
                    actual.siguiente.anterior = actual.anterior;
                } else {
                    cola = actual.anterior;
                }
            }
            actual = actual.siguiente;
        }
        System.out.println("Nodo con valor " + valor + " eliminado.");
        return;
    }
}

// Imprimir la lista de inicio a fin
public void imprimirAdelante() {
    Nodo actual = cabeza;
    System.out.print("Lista en orden: ");
    while (actual != null) {
        System.out.print(actual.dato + " ");
        actual = actual.siguiente;
    }
    System.out.println();
}

// Imprimir la lista de fin a inicio
public void imprimirAtras() {
    Nodo actual = cola;
    System.out.print("Lista en orden inverso: ");
    while (actual != null) {
        System.out.print(actual.dato + " ");
        actual = actual.anterior;
    }
    System.out.println();
}

public class Main {
    public static void main(String[] args) {
        ListaDoble lista = new ListaDoble();

        lista.agregar(10);
        lista.agregar(20);
        lista.agregar(30);

        lista.imprimirAdelante(); // Muestra: 10 20 30
        lista.imprimirAtras();   // Muestra: 30 20 10

        lista.eliminar(20);

        lista.imprimirAdelante(); // Muestra: 10 30
        lista.imprimirAtras();   // Muestra: 30 10
    }
}
```

